

CS 361 - Homework 04

Professor Mark W. Boady

1 Overview

In this homework you will simulate **Thread Race 2000**. This homework is all about communicate between threads and locks. We simulate a race between a collection of threads.

2 Thread Race 2000

Your program will take one command line input. It must be a integer and strictly larger than 0. The input is the number of threads that will compete in the race. Having 0 threads race would be pointless. Your program should produce an error if no value is given or an invalid input is given.

For example, to race three threads we we run `./bin/race 3`

For the remainder of this section, let us assume r is the number of racers. Your program will spawn $r + 1$ threads from main. Once they are all completed, your program will print the winners. One thread will be the **game master** who runs the race. The other r threads will be the racers.

The **Game Master** thread generates random dice rolls. The **racers** will then use these dice rolls to move across the race track. A **racer** wins when it has move 20 or more spaces. Once the **racer** is at the end of the race track it has crossed the finish line. Once all **racers** have crossed the finish line, the program is over. The **main** program should print a list of the threads ranked by when they crossed the finish line.

All threads will sleep during the race to simulate the program doing a complex calculation.

The basic logic of the **Game Master** thread is as follows:

```
1 While all threads have not crossed the finish line
2     Select a random number k between 0 and 5 (inclusive)
3     Roll k 6-sided dice (random number 1-6 inclusive)
4     Add the dice rolls to a queue
5     Sleep for a random length between 0-2 seconds
```

The basic logic of the **racer** thread is as follows:

```
1 While thread has not moved at least 20 spaces
2     Read a dice roll from the queue
3     Move forward that many spaces
4     Sleep for a random length between 0-2 seconds
```

You **may** use semaphores, locks, and/or conditional variables to manage the threads. You **may** use the semaphore class as provided in lecture as part of your solution. You must protect all the shared data structures. You **may not** use the thread safe queue class from class.

It is **absolutely** possible for one thread to grab multiple dice rolls before another thread executes based on how long it sleeps. This is allowed. As long as each thread sleeps for a random time between grabbing another dice roll form the queue, it may keep running forward. This is a race!

You may use the basic rand command or use the C++11 non-deterministic hardware entropy based random number generator.

You **must** use the queue from the standard library to pass the dice rolls between threads. (Do not build your own queue.)

You **must** make sure the following features are thread safe:

- I/O two threads should never write to cout at the same time - Dice Queue must be safe for the **game master** and **racer**

You **must** make sure to avoid the following:

- **Deadlock**: No threads can race forward (the program stops forever because no one can move) - **Starvation**: All threads must eventually finish the race (don't let one thread never get a dice roll)

Once you have completed your code, you will run a few tests and write your thoughts on how your code works. You will write your thoughts in a readme file.

Here is an example of how the race might execute. Please match the output style as closely as possible.

Note: Each Operating System decides thread IDs differently. They will be different between different computers and different runs.

```

1 ./bin/race 3
2 Starting Race with 3 threads.
3 Thread 140628519687936 has left the gate.
4 Thread 140628519687936 moved forward 2 spaces.
5 Thread 140628519687936 moved forward 6 spaces.
6 Thread 140628519687936 moved forward 1 spaces.
7 Thread 140628511295232 has left the gate.
8 Thread 140628502902528 has left the gate.
9 Thread 140628502902528 moved forward 5 spaces.
10 Thread 140628519687936 moved forward 4 spaces.
11 Thread 140628519687936 moved forward 4 spaces.
12 Thread 140628511295232 moved forward 5 spaces.
13 Thread 140628519687936 moved forward 2 spaces.
14 Thread 140628502902528 moved forward 4 spaces.
15 Thread 140628519687936 moved forward 1 spaces.
16 Thread 140628519687936 has crossed the finish line.
17 Thread 140628511295232 moved forward 3 spaces.
18 Thread 140628511295232 moved forward 3 spaces.
19 Thread 140628511295232 moved forward 3 spaces.
20 Thread 140628502902528 moved forward 2 spaces.
21 Thread 140628511295232 moved forward 1 spaces.
22 Thread 140628511295232 moved forward 2 spaces.
23 Thread 140628511295232 moved forward 5 spaces.
24 Thread 140628511295232 has crossed the finish line.
25 Thread 140628502902528 moved forward 1 spaces.
26 Thread 140628502902528 moved forward 6 spaces.
27 Thread 140628502902528 moved forward 4 spaces.
28 Thread 140628502902528 has crossed the finish line.
29 1: 140628519687936
30 2: 140628511295232
31 3: 140628502902528

```

2.1 Implementation

You are expected to write professional code. Use good variable and function names. Provide comments explaining how the code works. Document each function in the header file with comments. You will be graded on style as well as functionality.

2.2 Citations

If you use any outside resources, talk about algorithm design with other students, or get help on assignments you **must** cite your resources in the comments. Uncited sources are an Academic Honesty Violation. Cited sources may lead to a deduction depending on the amount of code used, but will not violate Academic Honesty Policies.

You are expect to write the majority of the code yourself and use resources for things like looking up commands. For example, if you forgot how to test if a file can be opened for reading you could look it up and cite a source. If you copy a critical algorithm and cite the code, you may still get a deduction for not developing the code yourself.

2.3 Readme

Your readme will include both instructions and reflections on your code. It must be stored on the root of your folder structure. It must include the following. Use markdown formatting and name your file readme.md.

There is no minimum or maximum length for the short essay questions, you are graded entirely on content. A short but comprehensive answer is better than a long confusing answer.

1. Your name and drexel ID (abc123@drexel.edu)
2. Instructions to run you code.
3. Short Essay Question 1: What semaphores did you use and why? (If you didn't use any, why not?)
4. Short Essay Question 2: What locks did you use and why? (if you didn't use any, why not?)
5. Short Essay Question 3: Why are you **confident** the program can never deadlock? (Explain how you programming it to avoid this)
6. Short Essay Question 4: Why are you **confident** the program can never let any thread starve? (Explain how you programming it to avoid this)
7. Short Essay Question 5: What was the most challenging part of this assignment?

2.4 Makefile

You **must** provide a makefile to compile your code. We will type make and it **must** build your program. We will the type make run and it must test your program. If there are any compile errors or a makefile is not provided we cannot test your code.

You must have the following make targets:

- make - Builds the Program
- make run - Runs three races with 2, 4, and 10 racers.
- make clean - Remove compiled code
- make doc - generate doxygen documentation

2.5 Other Requirements

If your submission does not meet the following guidelines we will not be able to grade it.

- You **must** use the C++ 17 Standard threads. No other thread libraries (pthreads, boost, etc) may be used. <https://en.cppreference.com/w/cpp/header/thread>
- Code **must** run on tux and be compiled with g++.

- All code **must** compile using the C++ 17 or above standard. (`--std=c++17`)
- All code **must** be submitted to blackboard learn as a zip.
- A working makefile **must** be provided.
- Must provide a readme file
- You may use libraries in the C++ 17 standard unless noted elsewhere in the instructions. <https://en.cppreference.com/w/cpp/header>
- Your code **must** compile. You should always submit stable code, we will not debug code that does not compile.

3 Grading

This homework is worth 100 points.

Question 1 : 2 points

Main Program - Handles Command Line Arguments

Question 2 : 2 points

Main Program - Starts all threads

Question 3 : 2 points

Main Program - Prints the final rankings at the end

Question 4 : 2 points

Main Program - Overall Style

Question 5 : 35 points

Racer Thread

Question 6 : 35 points

Game Master Thread

Question 7 : 2 points

Readme.md - Name and Email

Question 8 : 2 points

Readme.md - Instructions

Question 9 : 3 points

Readme.md - Short Essay 1

Question 10 : 3 points

Readme.md - Short Essay 2

Question 11 : 3 points

Readme.md - Short Essay 3

Question 12 : 3 points

Readme.md - Short Essay 4

Question 13 : 3 points

Readme.md - Short Essay 5

Question 14 : 2 points

Makefile is correct

Question 15 : 1 point

Zip Folder Formatted Correctly